МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования
«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ

ОТЧЕТ

о прохождении учебной (технологической (проектно-технологической)) практики

Обучающийся Хаметшин Ренат Рашидович, 11-303 _____
(ФИО, группа) (подпись)

Руководитель практики от КФУ:
старший преподаватель кафедры программной инженерии Шахова И.С.
(должность, ФИО)

Оценка за практику: _____ _____
(подпись руководителя практики)

Дата сдачи отчета: 30.12.2025

Казань – 2025 г.

Содержание

Введение **3**
1. Постановка задачи **4**
2. Обзор предметной области **6**
3. Обзор альтернативных решений **8**
4. Концепция программного решения **12**
Заключение **28**
Список использованных источников **30**

Введение

Учебная практика является одним из компонентов и составной частью учебного процесса студентов бакалавриата. Данный вид практики выполняет функции общепрофессиональной подготовки в части формирования необходимых компетенций и подготовки студентов к будущей профессиональной деятельности.

Дата начала практики: 01.09.2025 г.

Дата окончания практики: 30.12.2025 г.

Место прохождения практики: Институт информационных технологий и интеллектуальных систем.

Цель прохождения практики: Получение навыков поиска, формализации и проектирования решений проблем предметной области.

Для достижения поставленной цели практики были сформулированы следующие задачи:

1) Формулирование проблематики выбранной предметной области;
2) Анализ альтернативных решений сформулированной проблемы;
3) Описание концепции программного решения.

1. Постановка задачи

Начальная стадия разработки Android-приложений остаётся трудозатратной: значительное время уходит не на реализацию бизнес-логики, а на конфигурирование проекта, настройку Gradle, выбор архитектуры, подключение библиотек и инфраструктуры качества кода [1, 2]. Исследования показывают, что существенная доля времени на старте тратится на повторяющийся boilerplate-код и базовую настройку окружения, что приводит к замедлению вывода продукта на рынок и росту технического долга [3, 4].

При этом усложнение экосистемы Android и многообразие зависимостей повышают риск ошибок сборки: эмпирическое исследование открытых проектов фиксирует распространённые сбои, связанные с конфигурацией Gradle, несовместимостью версий зависимостей и проблемами окружения [2]. Ошибки этого класса часто возникают именно на этапе первоначальной настройки проекта, требуют значительных усилий на диагностику и отвлекают разработчиков от решения продуктовых задач [2].

Существующие средства ускорения старта - IDE-шаблоны, File Templates и готовые boilerplate-проекты - частично уменьшают объём ручной работы, но обычно жёстко задают стек и архитектуру, слабо поддерживают генерацию многомодульной структуры под конкретные требования продукта и требуют ручной донастройки дизайн-системы, CI и линтеров [2, 5, 6, 7, 8, 9]. В результате команды либо каждый раз «поднимают» инфраструктуру с нуля, либо копируют существующий проект, что ведёт к дублированию boilerplate-кода и неунифицированным конфигурациям между приложениями [1, 6, 10].

Таким образом, формируется конкретная проблема: отсутствует инструмент, который, опираясь на формализованное описание конфигурации (параметры Gradle, набор библиотек, архитектурный стиль, дизайн-система, CI и линтеры), автоматически генерирует целостный Android-проект с модульной архитектурой и продукт-специфичными элементами. Цель разрабатываемого в рамках курсового проекта решения - создать веб-систему

автоконфигурации Android-проектов, которая минимизирует ручной труд на начальной фазе разработки и обеспечивает воспроизводимый, стандартизированный каркас приложения на основе заданных параметров конфигурации.

2. Обзор предметной области

Предметная область работы связана с автоматизацией начальной конфигурации Android-проектов: генерацией структуры модулей, подключением зависимостей, заданием архитектурного шаблона и подготовкой CI/CD-окружения. Существенная часть времени на старте проекта расходуется на ручную настройку Gradle-скриптов, модулей и инфраструктуры, причём большая доля этого труда приходится на повторяющийся boilerplate-код, что снижает эффективность разработки [3, 11]. Отраслевые обзоры и исследования отмечают, что систематическая автоматизация таких рутинных операций позволяет уменьшить трудозатраты и количество ошибок начальной конфигурации проектов [11].

Один из базовых подходов в данной области - генерация кода на основе шаблонов (template-based code generation), при которой исходный код и конфигурационные файлы синтезируются из высокоуровневого описания проекта с использованием шаблонов и движка генерации [11]. Подобные решения реализованы в генераторах стартовых проектов в других экосистемах, например Spring Initializr для Java-приложений и Yeoman для веб-приложений, позволяющих через веб-интерфейс или консоль выбрать параметры и получить готовый проект [12, 13]. Для Android уже используются шаблоны Android Studio и пользовательские генераторы, однако они, как правило, жёстко привязаны к IDE и не предоставляют единого веб-интерфейса с централизованной конфигурацией архитектуры, зависимостей и CI/CD [8, 10].

Важной частью предметной области является модульная архитектура Android-приложений и практика разделения кода на независимые модули (core, data, domain, feature) для повышения масштабируемости и ускорения сборки. Руководства по архитектуре подчёркивают важность явного выделения слоёв и строгого управления зависимостями между ними, однако создание такой структуры на старте проекта чаще всего выполняется

вручную и требует опыта, что делает её перспективным объектом для автоматизированной генерации [14, 15].

Отдельное направление составляет автоматизация управления зависимостями и Gradle-конфигураций: вынос общих плагинов и версий библиотек в централизованные конфигурации, генерация build-скриптов и version catalogs по декларативному описанию технологического стека [16]. Близкую задачу решает автоматизация CI/CD для Android: исследования конфигураций open-source проектов показывают высокую вариативность пайплайнов для GitHub Actions, GitLab CI, CircleCI и отсутствие единых стандартов, при том что YAML-конфигурации чаще всего создаются и копируются вручную. Генерация готовых сценариев сборки, тестирования и публикации на основе параметров проекта рассматривается как способ снизить порог входа в CI/CD и уменьшить риск ошибок.

Наконец, в предметной области активно обсуждаются дизайн-системы и компонентные библиотеки (например, Material Components for Android) как источник значительного объёма повторяющихся ресурсов и кода интерфейса. Автоматическое создание UI-компонентов, ресурсных файлов и дизайн-токенов по заданной дизайн-системе позволяет стандартизировать внешний вид приложений и сократить объём ручной разметки и кода [17]. Совокупность работ по автоматизации boilerplate-кода, использованию шаблонов и внедрению инструментов генерации показывает устойчивый тренд на рост роли таких решений в повышении производительности команд и стандартизации архитектуры в мобильных проектах [3, 11].

3. Обзор альтернативных решений

Рынок программных инструментов для автоматической инициализации Android-проектов включает несколько решений, поддерживающих сразу современную архитектуру, интеграцию с Android Studio и опции настройки. Для анализа выбраны те альтернативы, которые реально покрывают потребности комплексной генерации проектов и используются в профессиональной разработке.

KMP Wizard (Kotlin Multiplatform Wizard) - официальный сайт от JetBrains для создания проектов, где Android является одной из целевых платформ [18, 19]. Генерирует структуру модулей по платформам, поддерживает Compose Multiplatform, работает из Android Studio и предоставляет базовый набор зависимостей (kotlinx-coroutines, kotlinx-serialization) [20, 21]. Однако отсутствует добавление своих зависимостей, их настройка, многомодульность бизнес-логики, генерация конфигураций DI, дизайн-системы и готовой настройки CI.

Compose Multiplatform Wizard - мастер для проектов с использованием Jetpack Compose Multiplatform, представляющий собой конфигуратор с открытым исходным кодом [22]. Поддерживает генерацию проектов с единой кодовой базой UI, заточен под современные подходы UI на Android. Однако не конфигурирует DI и CI, не реализует автоматическую генерацию компонент дизайн-системы и многомодульность, а также не настраивает зависимости автоматически.

QuickProjectWizard - плагин для Android Studio, предоставляющий шаблоны для генерации модульных Android-проектов [23, 24]. Поддерживает выбор архитектур, DI (Hilt, Koin), частично CI, создание модулей, может интегрировать линтеры и Compose, однако не предоставляет средств генерации продуктовых дизайн-систем.

Формулировка критериев и их обоснование:

Выбор критериев для анализа инструментов автоматической инициализации Android-проектов основан на практических и теоретических

требованиях к современным средствам генерации программных решений. Каждый критерий отражает одну из ключевых сторон профессиональной разработки мобильных приложений, влияющих на скорость внедрения, качество архитектуры и последующую масштабируемость системы.

1. Поддержка Android - базовый критерий, определяющий применимость решения к целевой платформе и обеспечение интеграции с нативными API Android Studio.
2. Генерация модулей (core/feature) фиксирует способность инструмента создавать многослойную архитектуру и обеспечивает структурное разделение бизнес-логики и интерфейсов.
3. Наличие шаблонов CI демонстрирует готовность решения к интеграции в континуум непрерывной сборки и доставки (CI/CD), сокращая трудозатраты на автоматизацию.
4. Конфигурация DI (Dagger/Hilt/Koin) оценивает возможность генерации инфраструктуры для внедрения зависимостей - ключевого механизма модульности и тестируемости.
5. Дизайн-система (токены, префиксы) отражает степень поддержки консистентного UI-оформления на уровне кодогенерации.
6. Boilerplate для слоёв data/domain характеризует уровень автоматизации низкоуровневых компонентов проекта и уменьшение рутинизации при старте разработки.
7. Интеграция линтеров (detekt, ktlint) обеспечивает качество кода и соблюдение код-стайла без дополнительных ручных настроек.
8. Поддержка Compose Multiplatform и многомодульная структура оценивают современность архитектурных подходов и готовность к расширению на разные платформы.
9. Наличие веб-интерфейса определяет удобство использования и доступность инструмента для командной работы без необходимости установки плагинов.

Совокупность этих критериев обеспечивает полноту оценки, охватывая все стадии жизненного цикла проекта - от генерации архитектуры до контроля качества и настройки CI. Такой набор признаётся необходимым и достаточным для построения эталонного решения, которое минимизирует количество ручных действий и обеспечивает готовность проекта к промышленной эксплуатации.

Критерий	**Мое решение**	**KMP Wizard**	**Compose Multiplatform**	**QuickProjectWizard**
Поддержка Android	есть	есть	есть	есть
Генерация модулей (core/feature)	есть	нет	нет	есть
CI конфиг по шаблону	есть	нет	нет	есть
Выбор DI (Dagger/Hilt/Koin)	есть	нет	нет	есть
Дизайн-система (токены, префиксы)	есть	нет	нет	нет
Boilerplate для data/domain слоя	есть	нет	нет	есть
Линтеры (detekt, ktlint)	есть	нет	нет	есть
Поддержка Compose Multiplatform	есть	есть	есть	есть

Многомодульная архитектура	есть	частично	частично	есть
Веб-интерфейс	есть	есть	есть	нет

Вывод: предложенное мной решение охватывает максимальное число функций из представленных альтернатив. Оно единственное обеспечивает поддержку всех ключевых направлений комплексной автоматизации (архитектура, зависимости и их настройка, DI, дизайн-система, линтеры, автоматизация CI) без необходимости ручной доработки после генерации проекта. В других инструментах ряд шагов требует дополнительной ручной настройки или не автоматизирован вовсе.

4. Концепция программного решения

Разрабатываемое программное решение представляет собой веб-систему автоконфигурации Android-проектов, которая в интерактивном режиме собирает требования разработчика и автоматически формирует готовый шаблон проекта с преднастроенной архитектурой, зависимостями и инфраструктурой сборки.

Концепция опирается на практику генераторов стартовых проектов (project scaffolding tools), таких как Spring Initializr [12], Kotlin Multiplatform Wizard [18], Compose Multiplatform Wizard [22] и QuickProjectWizard [23, 24], но адаптирована под специфику современных Android проектов.

Пользователь взаимодействует с системой через веб-интерфейс в формате пошагового мастера, разделённого на заголовок, боковую панель навигации и основную рабочую область, что соответствует рекомендуемым паттернам мастеров начальной конфигурации проектов в современных IDE и онлайн-генераторах.

Шесть последовательных этапов мастера соответствуют ключевым группам параметров конфигурации проекта и позволяют пользователю в явном виде контролировать архитектуру, технологический стек, пользовательский интерфейс и процессы качества по аналогии с существующими генераторами проектов, но с более детальной проработкой Android-специфичных аспектов.

Пользовательский сценарий работы системы:

На первом этапе пользователь вводит основную информацию о проекте (название приложения, идентификатор пакета, целевые платформы и версии SDK) (рис. 1), формируя блок метаданных будущего Android- или мультиплатформенного проекта по аналогии с Spring Initializr и Kotlin Multiplatform Wizard.

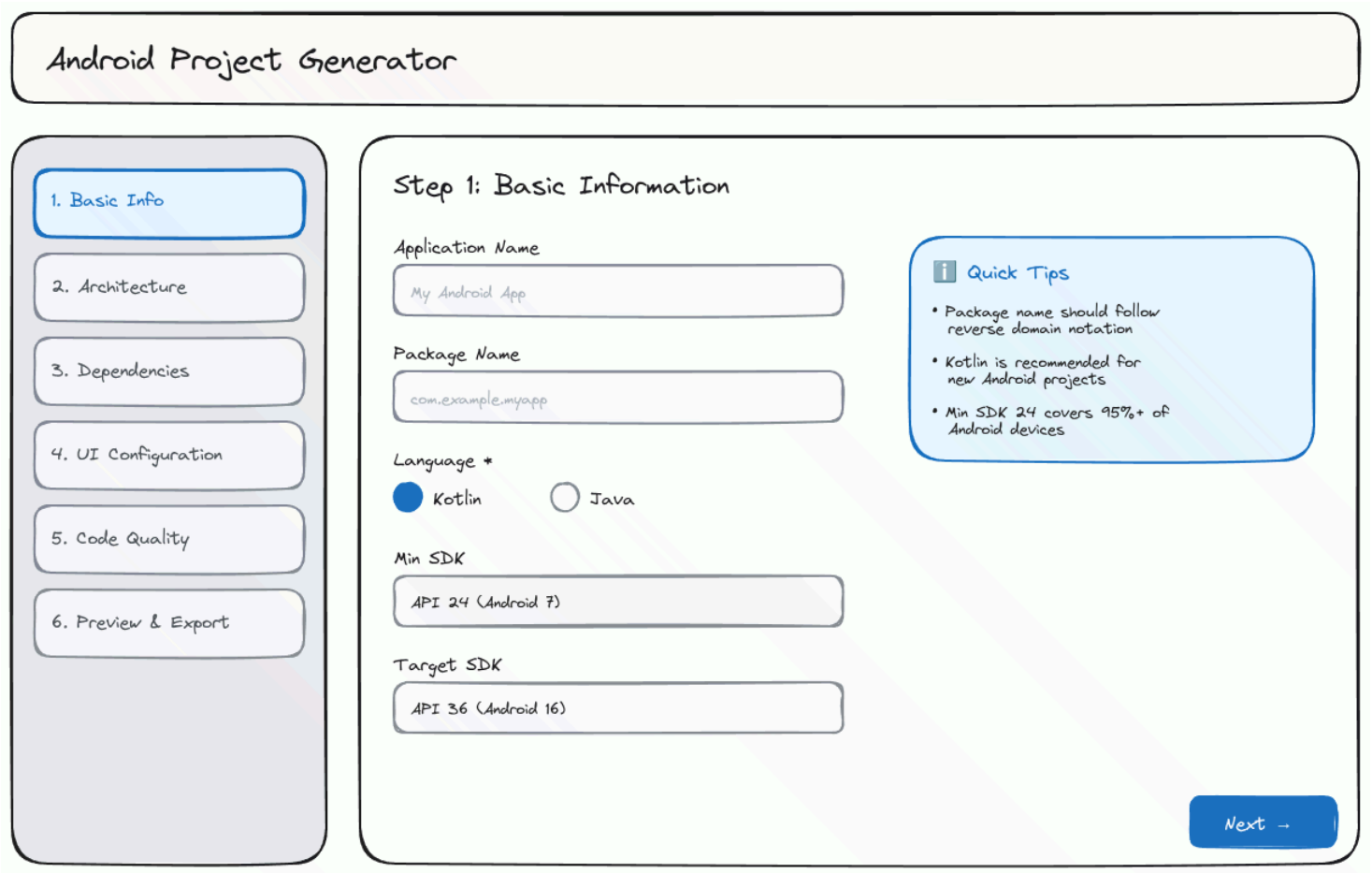

Рисунок 1 - экран этапа 1 мастера конфигурации с формой ввода основной информации о проекте: названия приложения, идентификатора пакета, целевых версий SDK.

На втором этапе пользователь определяет архитектурную модель проекта. Интерфейс данного этапа реализован в виде двух взаимоисключающих режимов, представленных вкладками «Preset Patterns» и «Custom», что позволяет охватить как типовые сценарии использования готовых архитектурных паттернов, так и потребности в создании нестандартных архитектурных решений (рис. 2).

Режим «Preset Patterns» предоставляет выбор из предустановленных архитектурных паттернов (MVVM, MVI, MVP) с заранее определённой структурой слоёв и межкомпонентных зависимостей, что соответствует наиболее распространённым практикам Android-разработки.

Режим «Custom» реализует концепцию Component Type Editor — интерфейса для декларативного описания произвольных типов архитектурных компонентов (рис. 3). В отличие от традиционных генераторов, ограниченных фиксированным набором сущностей (Screen, ViewModel, UseCase), данный режим позволяет пользователю определить собственную номенклатуру компонентов: Coordinator, Router, Interactor, Gateway, Bloc или иные типы, соответствующие архитектурным соглашениям конкретной команды. Для каждого типа компонента задаются следующие атрибуты: наименование, принадлежность к архитектурному слою (presentation, domain, data), шаблон генерируемого кода и перечень допустимых зависимостей.

Редактирование атрибутов типа компонента осуществляется посредством модального диалогового окна (рис. 4), которое содержит: поле наименования компонента, селектор архитектурного слоя, поле шаблона имени файла с поддержкой переменных подстановки (${Feature}, ${Package}), редактор шаблона исходного кода и набор элементов управления для указания допустимых межкомпонентных зависимостей.

Такой подход обеспечивает независимость системы от конкретных архитектурных парадигм и позволяет формализовать произвольные архитектурные соглашения в виде воспроизводимой конфигурации генератора.

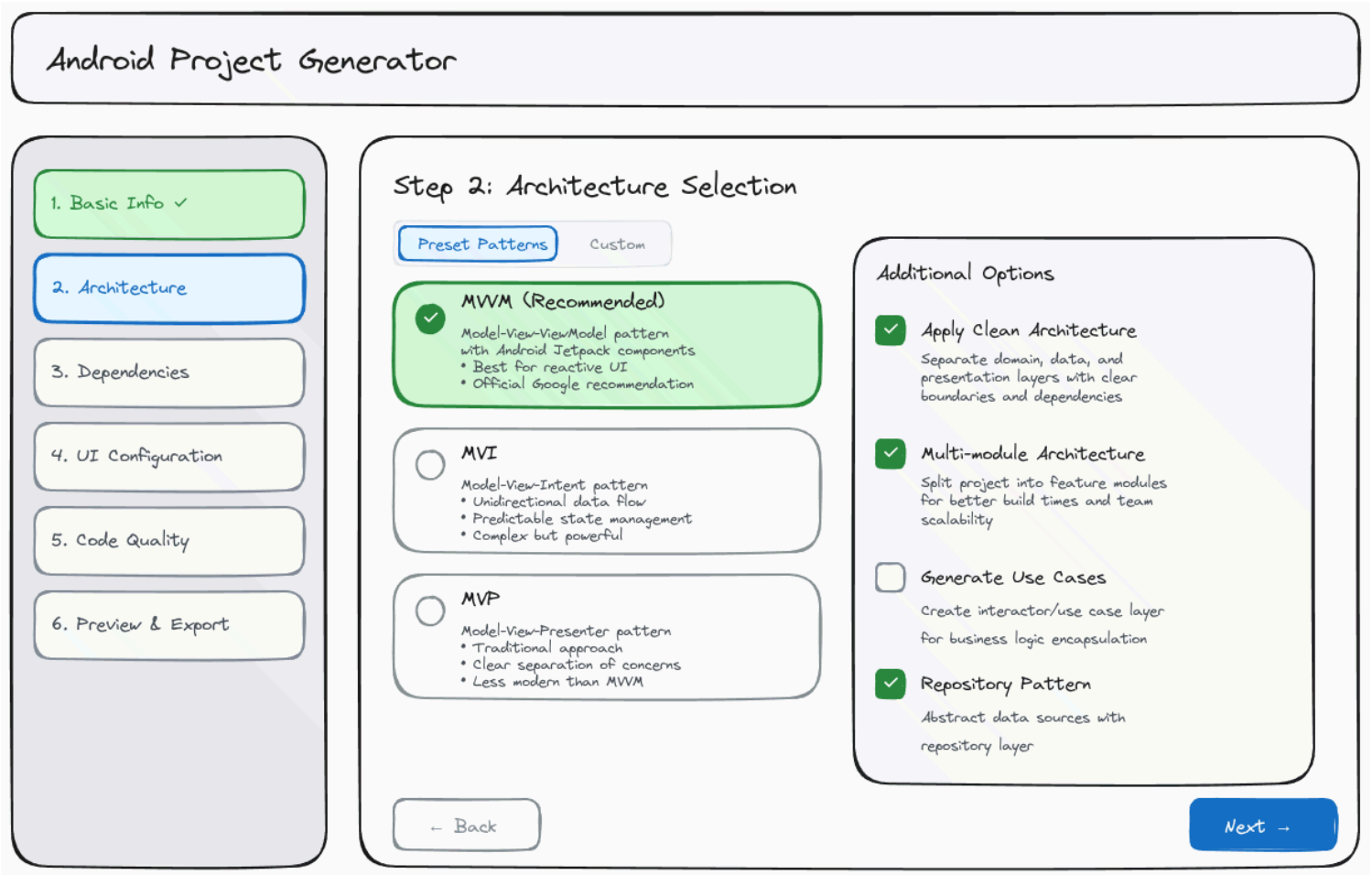

Рисунок 2 - экран этапа 2 мастера конфигурации: выбор архитектурной модели проекта с переключением между режимами «Preset Patterns» (предустановленные паттерны) и «Custom» (пользовательская архитектура).

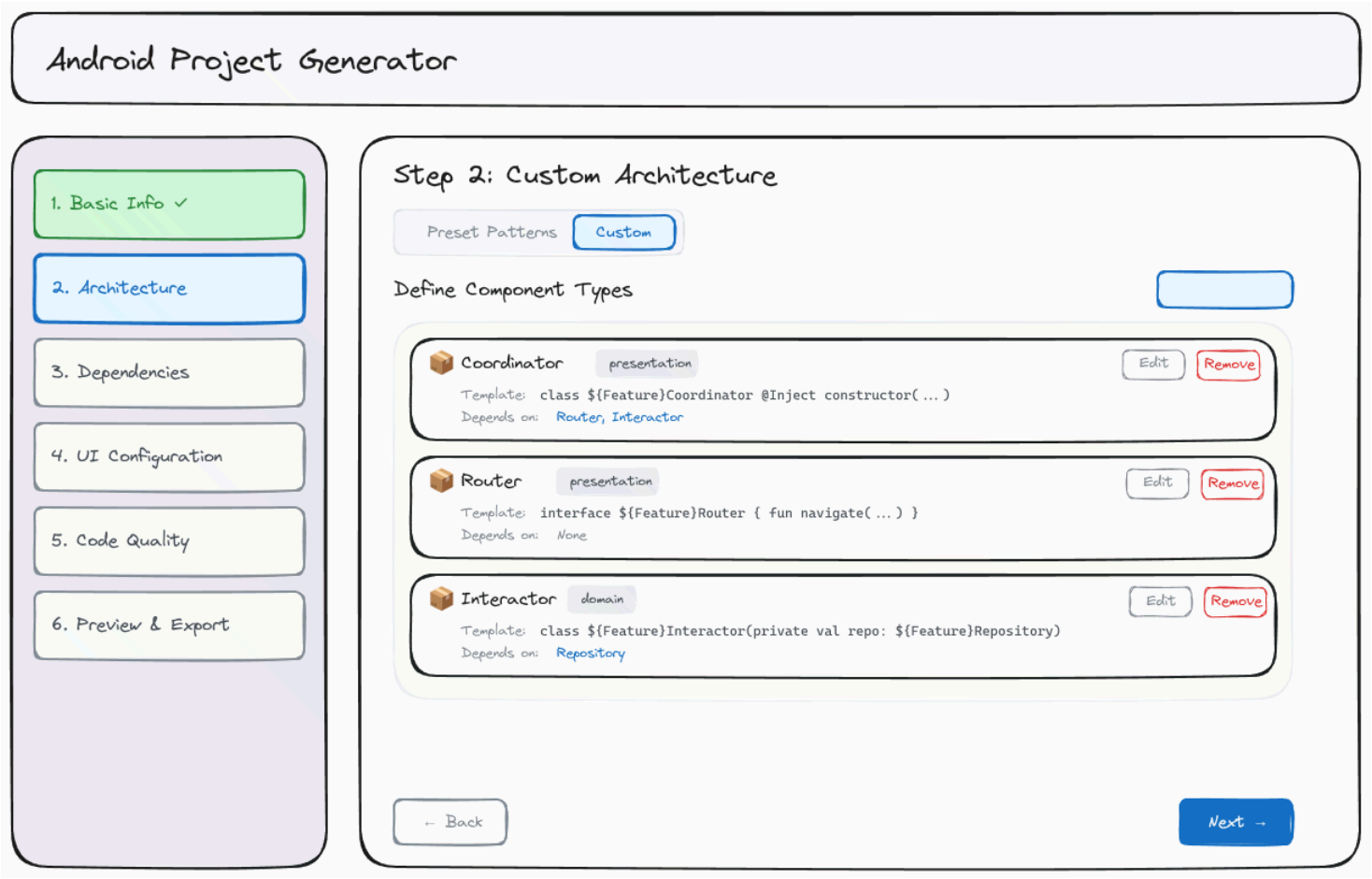

Рисунок 3 - экран режима «Custom» с интерфейсом Component Type Editor: список пользовательских типов компонентов с отображением наименования, принадлежности к архитектурному слою, шаблона кода и перечня зависимостей для каждого типа.

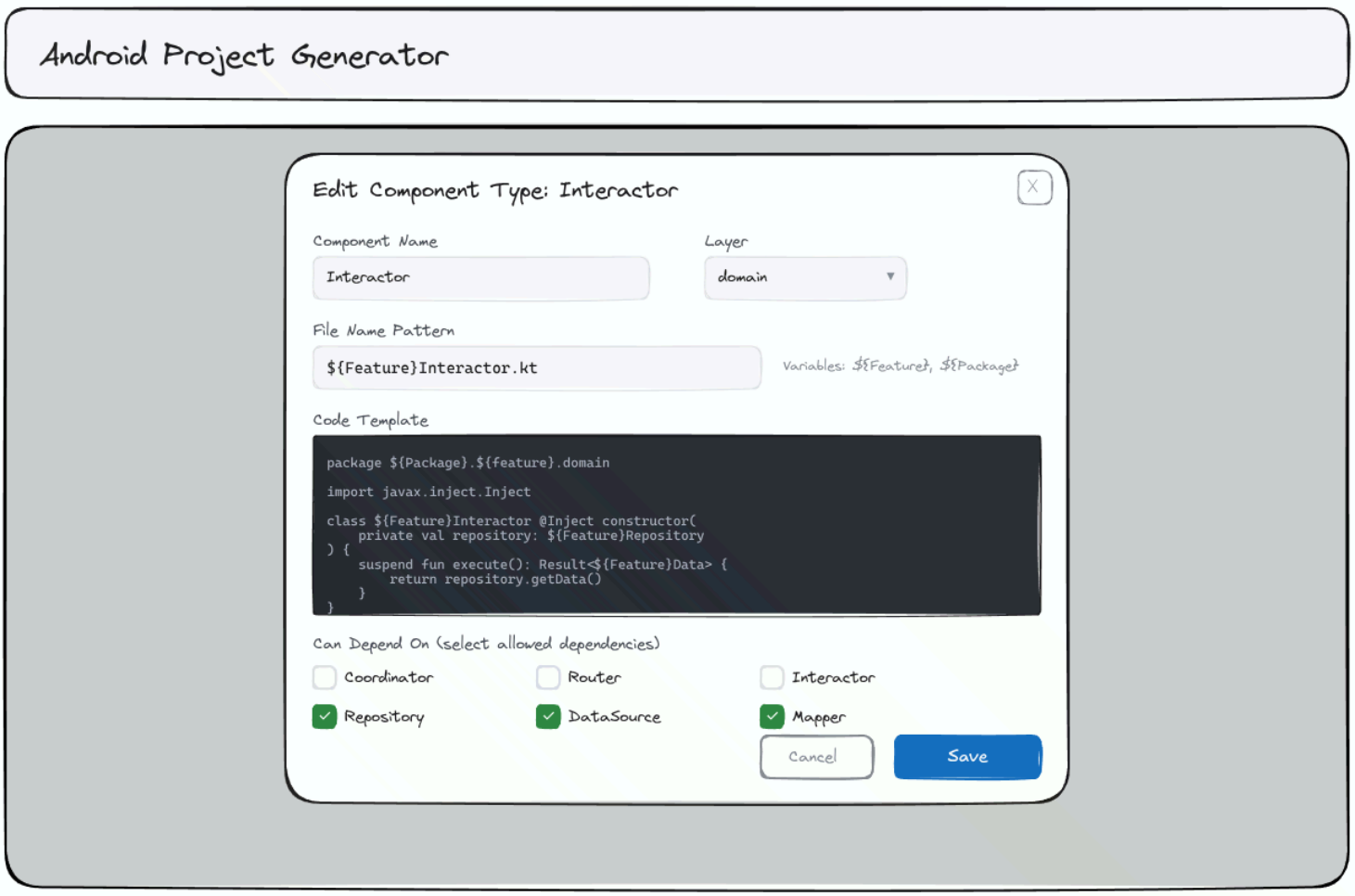

Рисунок 4 - модальное диалоговое окно редактирования типа компонента: форма ввода наименования, селектор архитектурного слоя, редактор шаблона имени файла и исходного кода, элементы управления для указания допустимых межкомпонентных зависимостей.

На третьем этапе реализован каталог зависимостей с поиском и фильтрацией по категориям (например, Network, Database), где пользователь отмечает необходимые библиотеки (Dagger/Hilt, Koin, Retrofit, Room, Coil, Timber, detekt, ktlint и др.) и их опциональные настройки. (рис. 5)

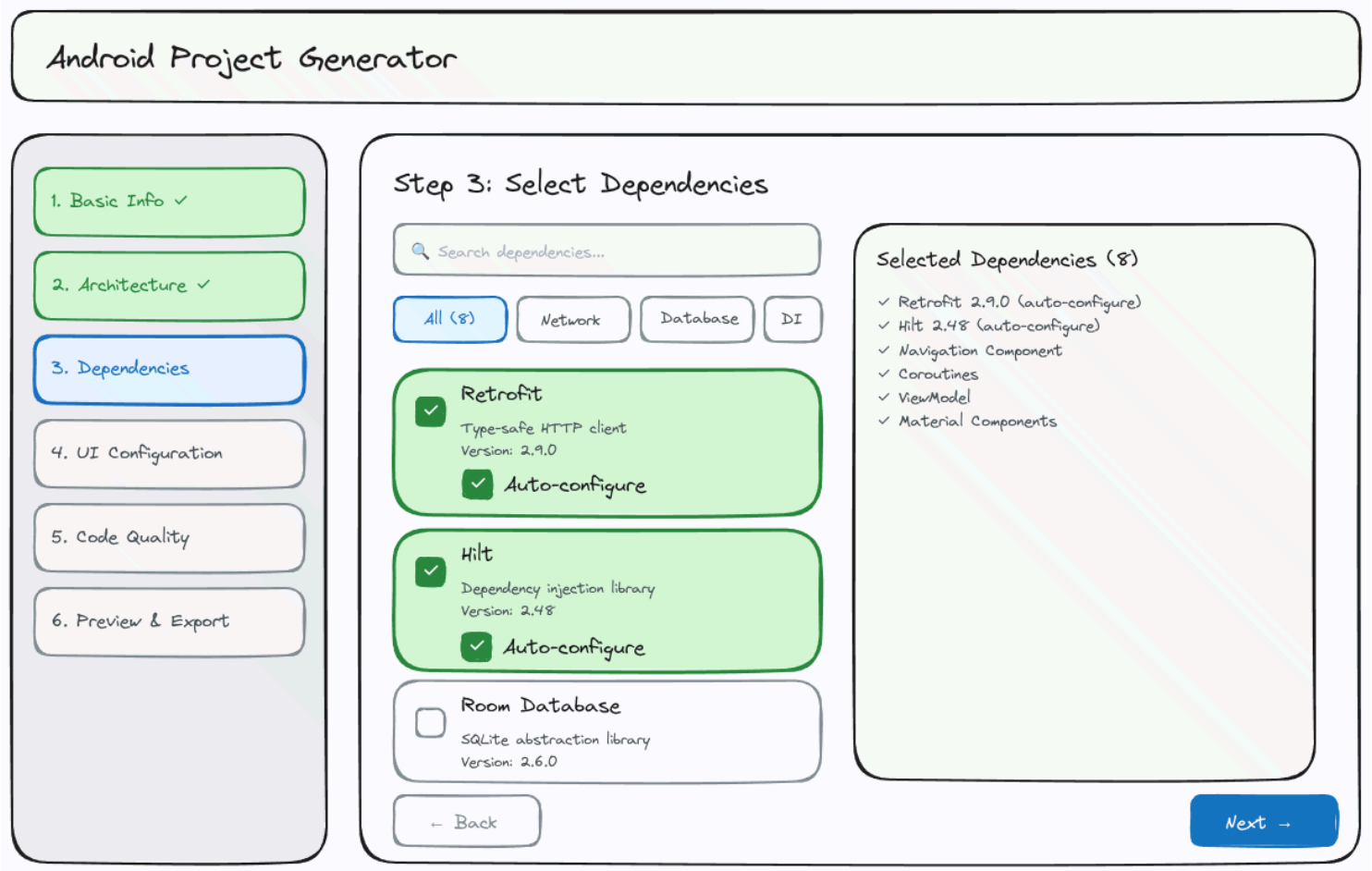

Рисунок 5 - экран этапа 3 мастера выбора зависимостей и технологического стека с панелью поиска, фильтрацией по категориям и отдельным списком уже выбранных библиотек.

На четвёртом этапе пользователь настраивает параметры пользовательского интерфейса (выбор UI-фреймворка, например, Jetpack Compose или XML-разметки, темы, цветовые палитры и префикса для дизайн-системы). (рис. 6)

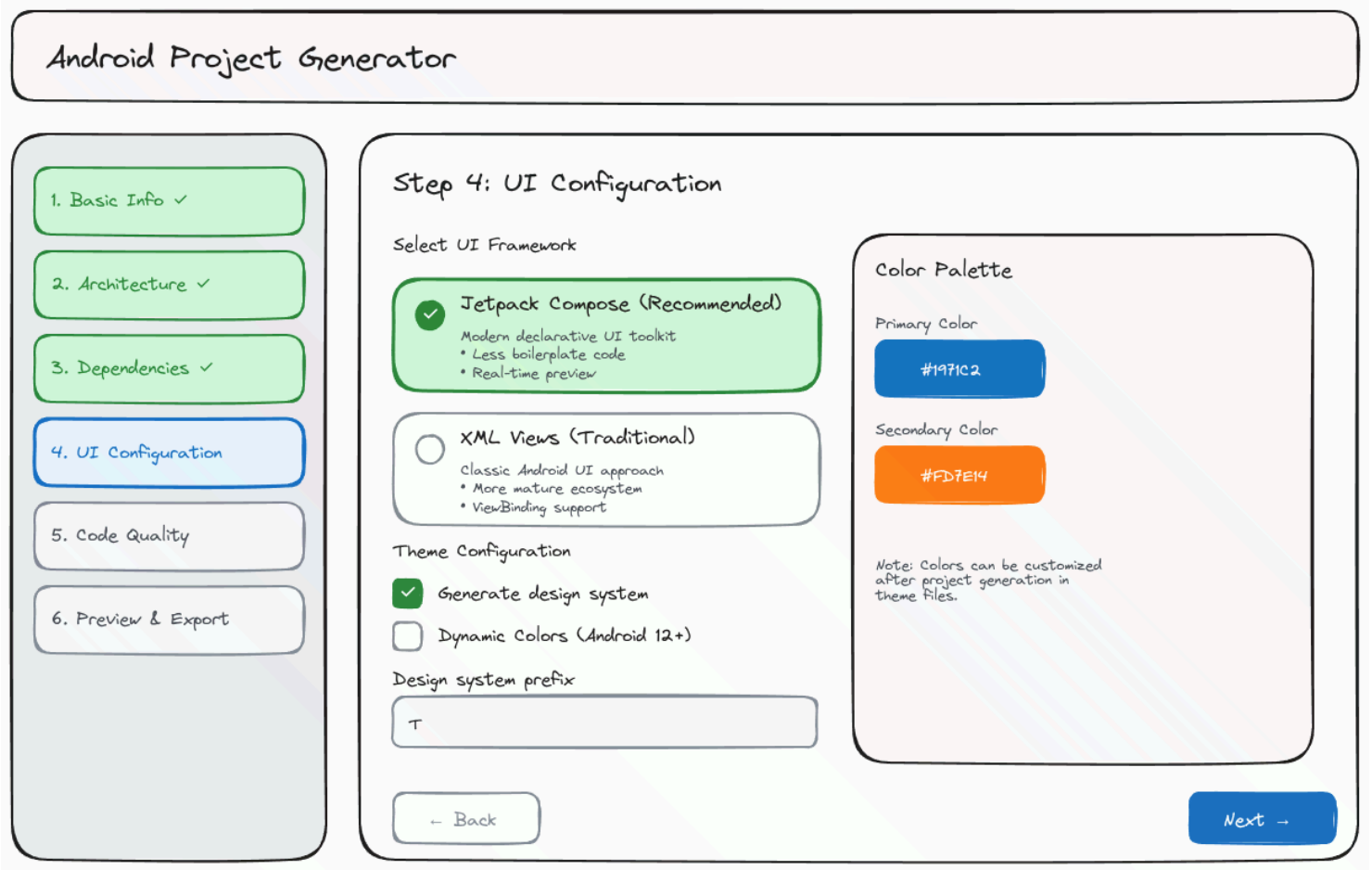

Рисунок 6 - экран этапа 4 мастера настройки пользовательского интерфейса с выбором UI-фреймворка, параметров темы, цветовой палитры и префикса для дизайн-системы.

На пятом этапе пользователь выбирает инструменты качества и CI/CD (линтеры, анализаторы кода, сценарии автоматической сборки и проверки качества), что согласуется с практикой современных Android-шаблонов и плагинов-мастеров. (рис. 7)

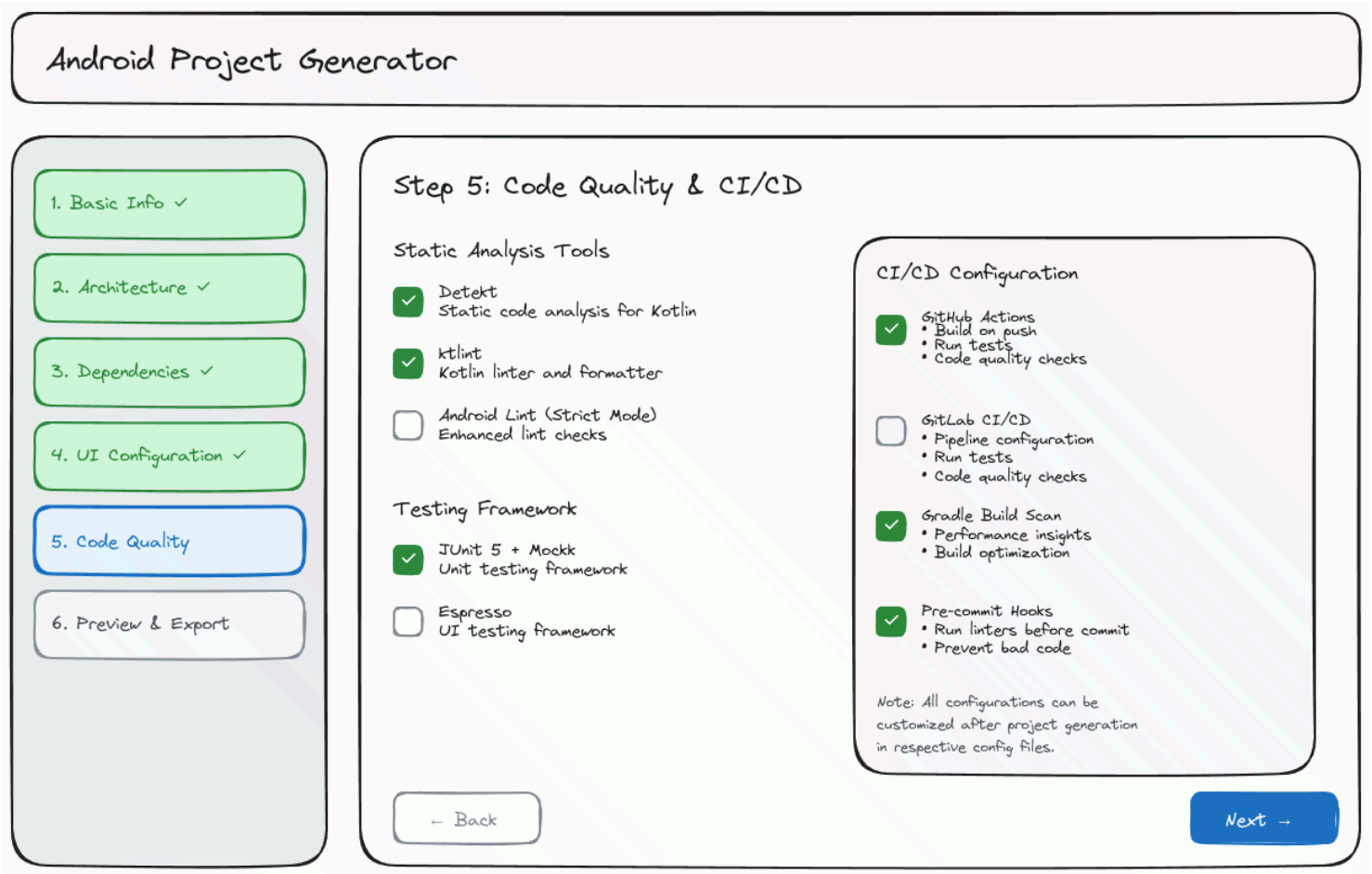

Рисунок 7 - экран этапа 5 мастера выбора инструментов качества и CI/CD с настройкой линтеров, анализаторов кода и шаблонов сценариев автоматической сборки и проверки

На шестом этапе система формирует предпросмотр структуры генерируемого проекта (набор модулей, каталогов и ключевых файлов) и предоставляет пользователю вкладки для просмотра содержимого отдельных файлов, а также опции экспорта результата (ZIP-архив проекта, ссылка для обмена). (рис. 8)

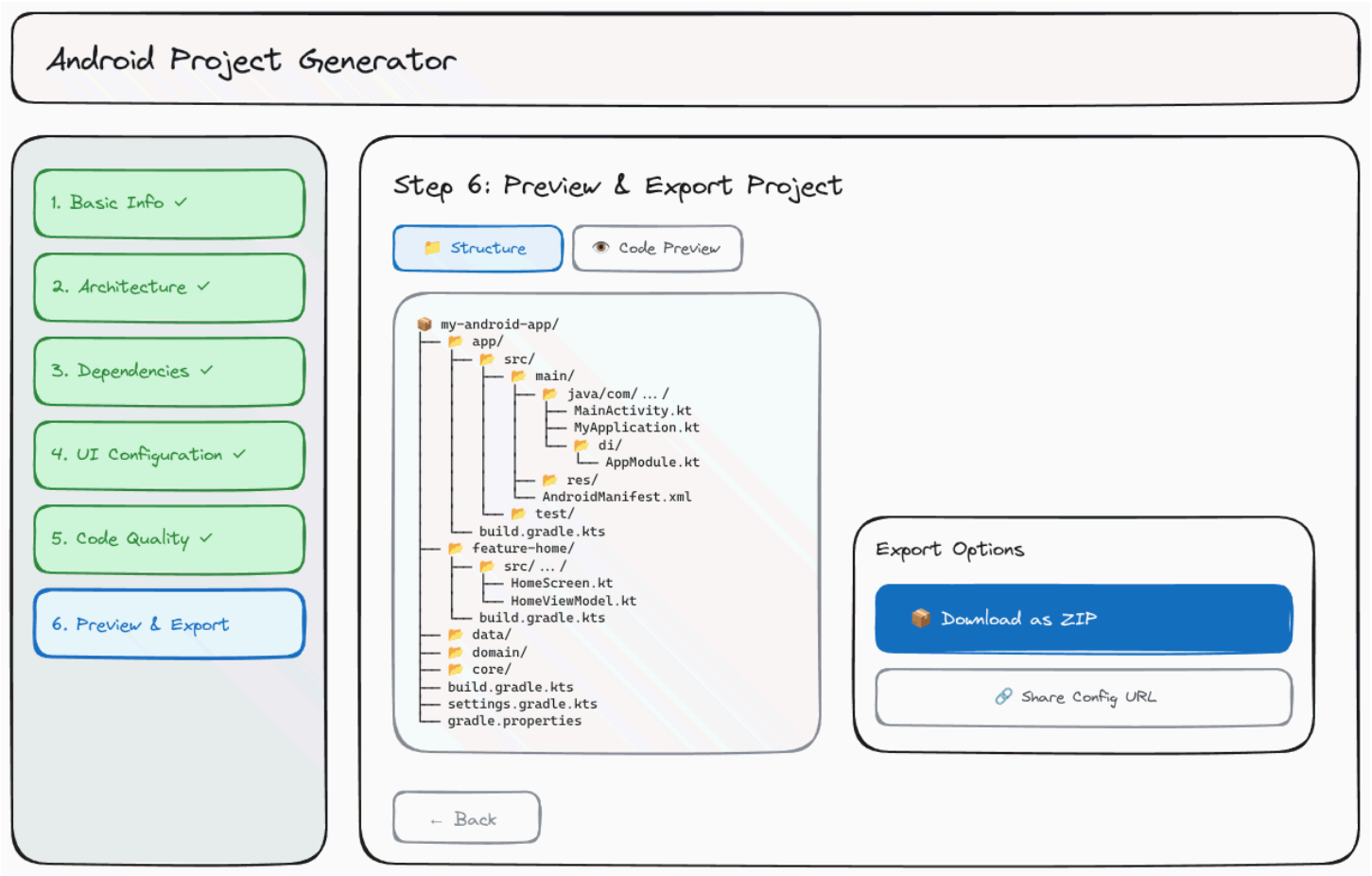

Рисунок 8 - экран этапа 6 мастера предпросмотра структуры сгенерированного проекта и параметров экспорта (скачивание ZIP-архива, копирование ссылки текущей конфигурации проекта).

На всех этапах мастер обеспечивает явную индикацию прогресса в боковой панели, поддержку навигации вперёд/назад и мгновенную обратную связь при выборе элементов, что соответствует требованиям к удобству и измеримой эффективности сценариев взаимодействия в интерфейсах разработки.

Структура входных данных:

- Входные данные в концепции решения представляют собой формально описанный конфигурационный объект, который агрегирует параметры, полученные на всех шести шагах мастера, и используется в качестве единственного источника правды для генератора проекта. С учётом сценария использования и UI-структуры данные логически

разбиваются на несколько групп, каждая из которых формируется на отдельном этапе мастера.

- Глобальные параметры проекта: название приложения, идентификатор пакета, набор целевых платформ, минимальная и целевая версии SDK, базовые параметры CI (GitHub Actions, GitLab CI и др.), выбранный архитектурный паттерн и уровень модульности.
- Параметры архитектурной модели: идентификатор режима конфигурации (preset или custom), а также зависящие от режима данные. Для режима preset сохраняется идентификатор выбранного паттерна (MVVM, MVI, MVP) и связанные с ним предустановленные параметры. Для режима custom формируется массив определений пользовательских типов компонентов, где каждый элемент включает: уникальный идентификатор типа, наименование, идентификатор архитектурного слоя, шаблон имени генерируемого файла с переменными подстановки, шаблон исходного кода компонента и множество идентификаторов типов, составляющих допустимый граф зависимостей. Данная структура обеспечивает полную формализацию произвольной архитектуры и её воспроизводимое применение при генерации проекта.
- Параметры зависимостей и технологического стека: список выбранных библиотек с их версиями, флагами включения/отключения и дополнительными опциями (генерация типовых DI-модулей, создание стандартного Retrofit-клиента, базовой схемы Room-БД, включение логирования Timber, активация и начальная конфигурация detekt/ktlint и других инструментов качества).
- Продукт-специфичные параметры: префикс компонентов дизайн-системы (например, TButton, TTextField), цветовая схема (primary/secondary) и выбор UI-фреймворка.
- Параметры процессов качества и доставки: набор активируемых линтеров и анализаторов, типы тестов и базовые конфигурации,

параметры CI/CD-пайплайна (шаблон workflow, ветки триггеров, шаги сборки и проверки), а также опции интеграции с внешними сервисами.

Все эти данные сериализуются в единую модель конфигурации, пригодную как для непосредственного запуска генерации, так и для последующего экспорта/импорта и повторного использования конфигурации для создания новых проектов.

Формирование каталога библиотек

Каталог библиотек, предлагаемых пользователю на этапе выбора зависимостей, формируется на основе курируемого реестра, поддерживаемого администраторами системы. Данный подход обусловлен необходимостью обеспечения совместимости предлагаемых зависимостей между собой и с целевыми версиями Android SDK, а также гарантии актуальности и безопасности включаемых библиотек.

Реестр библиотек организован в виде иерархической структуры с категоризацией по функциональному назначению: сетевое взаимодействие (Network), локальное хранение данных (Database), внедрение зависимостей (DI), обработка изображений (Image Loading), логирование (Logging), инструменты качества кода (Code Quality) и другие. Каждая запись реестра содержит: идентификатор библиотеки, наименование, описание функциональности, координаты артефакта в формате Maven (groupId:artifactId), рекомендуемую версию, перечень совместимых версий, список конфликтующих зависимостей и набор опциональных параметров конфигурации.

Для библиотек, требующих дополнительной настройки, реестр содержит метаданные генерации boilerplate-кода: шаблоны инициализации, типовые конфигурации DI-модулей, примеры использования. Например, для библиотеки Retrofit указываются шаблоны создания интерфейса API-клиента

и конфигурации HTTP-клиента; для Room — шаблоны базы данных, DAO-интерфейсов и сущностей.

Актуализация реестра осуществляется периодически на основе мониторинга официальных репозиториев (Maven Central, Google Maven Repository), анализа релизных циклов библиотек и оценки их совместимости с текущими версиями Android Gradle Plugin. Версионирование реестра позволяет обеспечить воспроизводимость генерации проектов и возможность отката к предыдущим конфигурациям при необходимости.

Такой подход к формированию каталога библиотек обеспечивает баланс между широтой выбора и надёжностью генерируемых проектов, минимизируя риск возникновения конфликтов зависимостей и ошибок сборки на начальном этапе разработки.

Алгоритмы обработки конфигурации:

Концепция решения предусматривает многоэтапную обработку пользовательской конфигурации: от первичной валидации и построения внутренней модели проекта до генерации файлов на основе шаблонов и упаковки результата в удобный для разработчика формат.

Логика обработки изначально проектируется модульной, чтобы поддерживать расширение набора зависимостей, архитектур и выходных артефактов без изменения базового конвейера генерации.

Основные этапы обработки конфигурации включают (рис. 9):

- Валидация входных данных - проверка корректности имени пакета, совместимости версий платформы и зависимостей, согласованности выбранной архитектуры и модульной структуры, а также полноты критически важных полей (название проекта, минимальная версия SDK и др.).
- Формирование внутренней модельной конфигурации - построение абстрактного описания проекта, включающего структуру модулей,

набор зависимостей, список генерируемых файлов (boilerplate-кода), карту параметров подстановки для шаблонов и правила привязки элементов дизайн-системы к выбранным фреймворкам UI.

- Генерация проекта на основе шаблонов - применение template-движка, который, используя модель конфигурации, генерирует корневые файлы (settings.gradle, корневой build.gradle), структуру каталогов, модули (app, core, features) и код для интеграции выбранных библиотек и архитектурных паттернов.
- Постобработка и упаковка результата - выполнение финальных шагов (форматирование файлов, подготовка структуры для импорта в Android Studio) и формирование ZIP-архива проекта или альтернативных форматов экспорта.

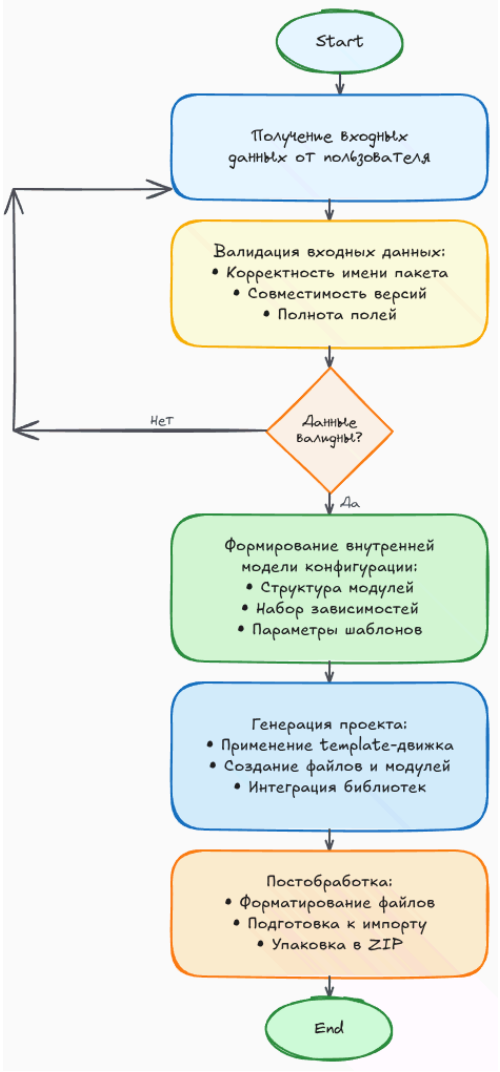

Рисунок 9 - UML-диаграмма активности процесса генерации Android-проекта

Такой конвейер позволяет использовать единый конфигурационный объект для генерации проектов различной сложности и назначения, обеспечивая воспроизводимость и стандартизированность стартовой архитектуры и инфраструктуры.

Выходные данные и результат работы системы:

Результатом работы системы является полностью сгенерированный проект, который пользователь может сразу открыть в Android Studio или другой совместимой среде разработки, минуя рутинный этап ручной настройки структуры и зависимостей.

Ключевые выходные данные включают:

- Структуру Gradle-модулей, соответствующую выбранной архитектуре (например, app, core:ui, core:designsystem, features:<feature>:data, features:<feature>:domain), с уже настроенными межмодульными зависимостями.
- Конфигурацию системы сборки: корневые и модульные build.gradle-файлы, плагины, репозитории, подключённые линтеры и инструменты анализа качества, а также заготовки конфигураций CI/CD-пайплайна.
- Boilerplate-код архитектурных компонентов, генерируемый в соответствии с выбранным режимом конфигурации. В режиме custom система синтезирует файлы компонентов на основе пользовательских шаблонов кода с автоматической подстановкой контекстных переменных (наименование функциональности, идентификатор пакета, перечень зависимостей). Генерируемый код учитывает заданные ограничения межкомпонентных зависимостей, что обеспечивает соблюдение архитектурных инвариантов проекта. В режиме preset

формируются стандартные компоненты выбранного паттерна с типовой структурой и зависимостями.

- Набор продукт-специфичных компонентов (UI-компоненты с заданным префиксом, токены дизайн-системы, базовые темы и цветовые схемы), согласованный с выбранным UI-фреймворком.
- Экспортируемый ZIP-архив готового проекта и/или файл конфигурации мастера, который может быть повторно использован или передан коллегам, что концептуально согласуется с подходом существующих генераторов проектов, но с учётом Android-ориентированной специализации.

Таким образом, концепция программного решения задаёт целостный цикл: от пошагового сбора параметров через специализированный веб-мастер до автоматической генерации стандартизированного Android-проекта, который учитывает архитектурные и инфраструктурные требования, предъявляемые к современным мобильным и мультиплатформенным приложениям.

Заключение

Данная практика была направлена на формирование следующих компетенций:

Шифр компетенции	Расшифровка приобретаемой компетенции
ОПК-2	Способен понимать принципы работы современных информационных технологий и программных средств, в том числе отечественного производства, и использовать их при решении задач профессиональной деятельности
ОПК-3	Способен решать стандартные задачи профессиональной деятельности на основе информационной и библиографической культуры с применением информационно-коммуникационных технологий и с учетом основных требований информационной безопасности
УК-6	Способен управлять своим временем, выстраивать и реализовывать траекторию саморазвития на основе принципов образования в течение всей жизни

В ходе выполнения курсовой работы была разработана концепция веб-приложения для автоматизированной генерации Android-проектов с настраиваемой архитектурой и зависимостями.

Проведен анализ современных подходов к разработке Android-приложений, изучены архитектурные паттерны (MVVM, MVI, MVP) и популярные библиотеки.

Спроектирован пользовательский интерфейс в виде пошагового мастера конфигурации из шести этапов: базовая информация, архитектура, зависимости, UI-настройки, инструменты качества кода и экспорт проекта.

Определена структура генерируемых проектов с поддержкой модульной архитектуры и архитектурных паттернов.

Все пункты индивидуального задания выполнены в полном объеме. Разработанная система включает все запланированные функции: пошаговый мастер, выбор архитектуры и зависимостей, настройку UI и экспорт проекта. Дополнительно созданы wireframe-макеты всех экранов.

В процессе работы освоены навыки проектирования пользовательских интерфейсов, разработки архитектуры веб-приложений и составления технической документации. Основные трудности были связаны с необходимостью глубокого изучения экосистемы Android-разработки.

Работа выполнена успешно в установленные сроки. Разработанная концепция соответствует современным требованиям и учитывает актуальные тренды. Проект имеет практическую ценность и может быть развит до полнофункционального продукта.

Список использованных источников

1. Barnett S., Vasa R., Grundy J. Bootstrapping Mobile App Development // 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. IEEE, 2015. P. 657–660.
2. Bodepudi L.P. et al. Diagnosing and Resolving Android Applications Building Issues: An Empirical Study // arXiv. 2025.
3. 😍 Discover over 28+ Mobile Boilerplates 🚀 [Электронный ресурс] // boilerplatelist.com. 2025. URL: https://boilerplatelist.com/mobile-boilerplates/ (дата обращения: 12.11.2025).
4. Why Developers Waste 40% Time on Boilerplate Code [Электронный ресурс] // The SSS AI. 2025. URL: https://thesss.ai/blog/the-sss-ai-impact-1/the-hidden-cost-of-project-kickstarts-why-developers-spend-40-of-their-time-on-boilerplate-code-11 (дата обращения: 12.11.2025).
5. Denys. The APP Solutions' Boilerplate for Android Development [Электронный ресурс] // The App Solutions. 2025. URL: https://theappsolutions.com/blog/development/tas-android-boilerplate/ (дата обращения: 03.11.2025).
6. Schiar Junior G.L.S.J. Criação de uma solução arquitetônica para organização de código em aplicações Android: Thesis / dissertation. Universidade Federal de Santa Catarina, 2019.
7. Android IDE Template Format [Электронный ресурс] // Habr. 2016. URL: https://habr.com/ru/articles/274959/ (дата обращения: 12.11.2025).
8. Как не писать лишнего [Электронный ресурс] // Habr. 2016. URL: https://habr.com/ru/articles/317970/ (дата обращения: 12.11.2025).
9. Уменьшаем написание boilerplate с помощью File Templates [Электронный ресурс] // Habr. 2025. URL: https://habr.com/ru/companies/tbank/articles/872684/ (дата обращения: 12.11.2025).
10. Tuan J. How Long Does It Take To Build an App in 2025

[Электронный ресурс] // Topflight Apps. 2025. URL: https://topflightapps.com/ideas/how-long-does-it-take-to-develop-an-app/ (дата обращения: 12.11.2025).

11. Syriani E., Gray J., Vangheluwe H. Modeling a Model Transformation Language // Domain Engineering. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013. P. 211–237.

12. spring-io. GitHub - spring-io/initializr: A quickstart generator for Spring projects [Электронный ресурс] // GitHub. URL: https://github.com/spring-io/initializr (дата обращения: 15.11.2025).

13. The web's scaffolding tool for modern webapps [Электронный ресурс] // Yeoman. URL: https://yeoman.io/ (дата обращения: 15.11.2025).

14. Guide to Android app modularization [Электронный ресурс] // Android Developers. URL: https://developer.android.com/topic/modularization (дата обращения: 15.11.2025).

15. Gradle Documentation [Электронный ресурс] // Gradle. URL: https://docs.gradle.org/current/userguide/organizing_gradle_projects.html (дата обращения: 15.11.2025).

16. Ghaleb T., Abduljalil O., Hassan S. CI/CD Configuration Practices in Open-Source Android Apps: An Empirical Study // ACM Transactions on Software Engineering and Methodology. 2025.

17. Material Components [Электронный ресурс] // Android Developers. URL: https://developer.android.com/design/ui/mobile/guides/components/material-overview (дата обращения: 15.11.2025).

18. Kotlin Multiplatform Wizard [Электронный ресурс] // JetBrains. URL: https://kmp.jetbrains.com/ (дата обращения: 16.11.2025).

19. Volodko E. Kotlin Multiplatform Tooling: Now in IntelliJ IDEA and Android Studio [Электронный ресурс] // The JetBrains Blog. 2025. URL: https://blog.jetbrains.com/kotlin/2025/05/kotlin-multiplatform-tooling-now-in-intellij-idea-and-android-studio/ (дата обращения: 16.11.2025).

20. Setup your environment (Kotlin Multiplatform) [Электронный ресурс] // Android Developers. URL: https://developer.android.com/kotlin/multiplatform/setup (дата обращения: 16.11.2025).

21. Adding dependencies on multiplatform libraries [Электронный ресурс] // Kotlin Multiplatform Help. URL: https://kotlinlang.org/docs/multiplatform/multiplatform-add-dependencies.html (дата обращения: 16.11.2025).

22. terrakok. GitHub - terrakok/Compose-Multiplatform-Wizard: Compose Multiplatform Wizard [Электронный ресурс] // GitHub. URL: https://github.com/terrakok/Compose-Multiplatform-Wizard (дата обращения: 16.11.2025).

23. cnrture. GitHub - cnrture/QuickProjectWizard: 🚀 Ultimate Android development companion for Android Studio. Create projects with modern dependencies, generate modules & features, and access essential dev tools - all in one place! [Электронный ресурс] // GitHub. URL: https://github.com/cnrture/QuickProjectWizard (дата обращения: 16.11.2025).

24. QuickProjectWizard. QuickProjectWizard [Электронный ресурс] // QuickProjectWizard. URL: https://quickprojectwizard.candroid.dev/overview (дата обращения: 16.11.2025).